Ui: a Unicon Development Environment

Clinton L. Jeffery, Hani Bani-Salameh, Sean Harris, Ben Jeffery, Shea Newton, and Serendel Macphereson

Unicon Technical Report: 12a

2015-05-22

Abstract

Ui is a simple integrated development environment (IDE) for the Unicon programming language. Ui makes it simple to edit, compile, run, and debug Unicon programs. Ui runs on any platform that supports Unicon's 2D graphics facilities, including UNIX-based and Windows platforms. Its source code provides various examples of extending and customizing a graphical user interface generated using the IVIB interface builder tool.

Unicon Project
http://unicon.org
University of Idaho
Department of Computer Science
Moscow, ID, 83844, USA

1 1. Introduction

This report describes a simple integrated development environment for the Unicon programming language[1], called Ui. The goal for Ui is to make it as simple as possible to edit, compile, and execute Unicon programs. By design and intention, Ui is much smaller and easier to learn than industry behemoths such as Eclipse and Visual Studio.

Even Ui's very "simple" goals are complicated by the fact that the tool needs to run on any machine where Unicon runs with graphics facilities enabled. At this time, this primarily consists of UNIX-based systems that run the X Window System, such as Linux, Solaris, Mac OS X, and Microsoft Windows-based machines. There is a practical minimal screen resolution limit, probably around 800x600 for this tool.

Ui is a product of the Unicon Project and is a standard part of the Unicon programming language distribution, developed under the GPL, hosted on Source Forge, and downloaded from http://unicon.org. If you are not already familiar with Unicon you may wish to consult other technical reports from that site, such as UTR #7[2], along with this one. Windows Unicon and the Ui environment are based on the volunteer work of many people; final responsibility for this release rests with Clinton Jeffery of University of Idaho. Send requests, and bug reports to `jeffery@cs.uidaho.edu`.

2 2. Editing, Compiling, and Executing Programs

Double-click the Windows Unicon icon to launch *Ui*, the Unicon IDE (integrated development environment). *Ui* is written in Unicon and allows you to edit, compile, and execute programs from a graphic user interface. Ui is designed to run the same on UNIX and Linux as it does on Windows. Ui's documentation (this file) may be accessed on-line through its Help menu. To start, you must select the name of a file to edit, in following dialog:

| | Figure 1: Opening a File within Ui

You can easily select an existing Unicon source file, or name a new one. If you click "Open" without choosing a name, you will be given the default name of "noname.icn". Unicon source files generally must use the extension `.icn` and should be plain text files without line numbers or other extraneous

information. Editing your program occurs within the main *Ui* window, which might look like this:

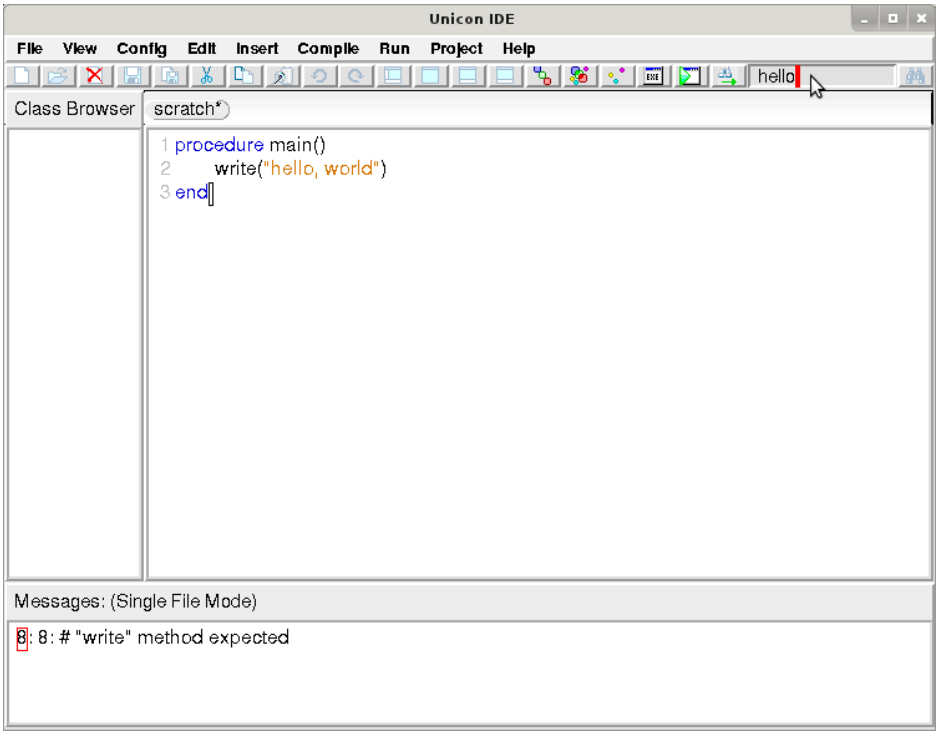
| | Figure 2: Ui's Editing Interface

The top area shows program source code, while the bottom portion shows messages such as compiler errors. You can change the font and the number of lines used to show messages from the Edit menu.

When you are done editing your program, you can save it, compile it, or just "make" (save, compile and link an executable) and run your program with menu options. The Arguments command in the Run... menu let's you specify any command-line arguments the program should be given when it is executed.

3 3. Find/Replace

Ui's Find/Replace capabilities include a search text-field on the right side of the tool-bar for quickly finding text, and an expanded dialog for searching and replacing.



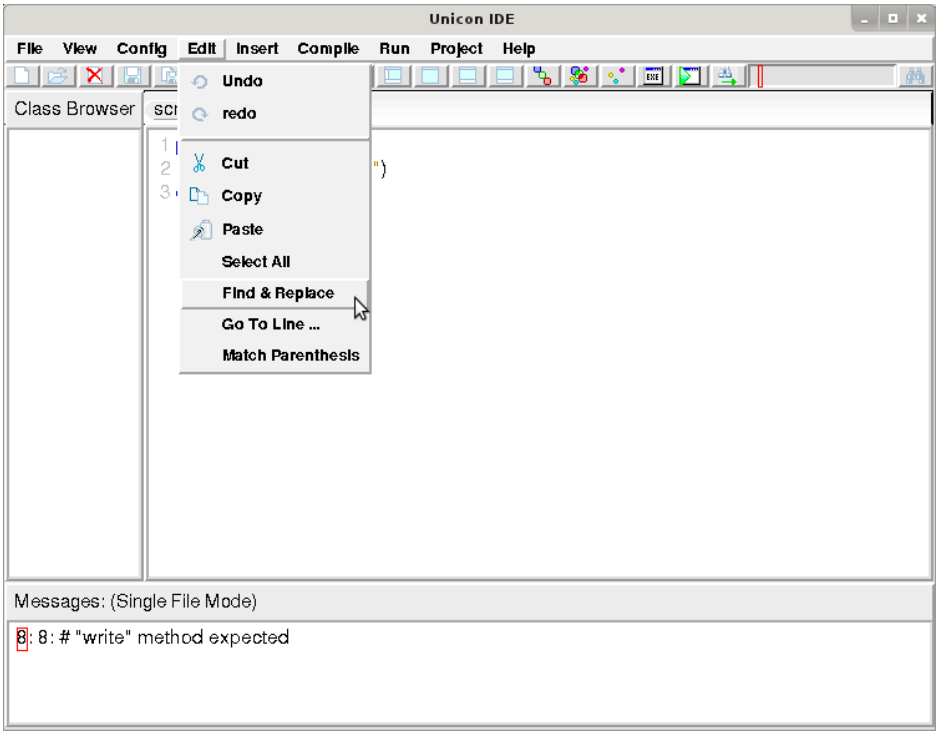


Figure 3: Search from the Toolbar (left) or Find & Replace from the Edit Menu (right) The search text-field is bracketed by two buttons: a find button on the far right edge which initiates a search, and a direction toggle button to the left of the search text-field that changes the direction of the search.

For more substantial search tasks, the Find & Replace dialog accessed from the Edit menu supports case-insensitive, backwards, and regular expression searching.

| | Figure 4: the Find and Replace Dialog

4 4. Error Handling

Compile errors result in a message in which the editor highlights the line at which the error was detected, like this:

| | Figure 5: Reporting Compiler Errors in Ui

Run-time errors also result in a message for which the source line is highlighted. The message for a run-time error includes Unicon's standard traceback of procedures from `main()` to the procedure in which the error

occurred. When the error messages get long, you can either increase the number of lines for the message window (as was done here) or scroll through the message window's entire text using the scrollbar.

| | Figure 6: A Run-time Error in Ui. The message window's size is adjustable.

5 5. Projects

Editors note: unfinished work-in-progress. This might constitute the specification for some future intended functionality rather than documentation of the current behavior. You have been warned.

Ui works on programs comprised of one or more source files. A *project* is Ui's abstraction of a program, consisting of the program name, working directory in which it is built, and set of source files and their dependencies on other files. By default, Ui takes it that it is working on single file project consisting of the current source file. If you are working on a single-file project, and you open a new .icn source file, Ui switches the editor and compile and link commands to work on this new program.

Ui project files are Makefiles, given the extension .Makefile. The format of Makefiles is described in documentation for one of the widely used "make" programs such as GNU make (see for example www.gnu.org/software/make/_). You can write your makefiles by hand, but in order for Ui to keep things straight, you had better stick closely to the format that it generates for you when you create a New Project. The dialog for creating a project let's you specify the set of source files and their dependencies for a Unicon program.

| | Figure 7: the Project Dialog

When you open a project file, Ui goes into "project mode", and adds the source files and their dependencies to your Class Browser allowing you to switch easily between files relevant to your project. If you subsequently open a source file not in the project, Ui asks if you want to add that source file to the project, or edit that file as a separate program. In general, project files allow you to "make" large projects efficiently. Underneath the covers, Ui invokes the other Unicon program executables to do the work of compiling and running programs, described below.

When Ui "makes" a program executable, it recompiles those modules listed in the project file whose modified time is newer than their corresponding object files. Ui does not present consider link declarations embedded in

source files, which generally are used for library modules; recompilation will not be triggered by such dependencies. When you use Ui, you should generally use link directives for library files (such as Unicon Program Library modules), and use explicit project file entries for all of your own source files. Files detailed in the .Makefile must not also be referenced in a link statement; linking the same module twice causes link errors. For projects, the executable that is produced and the project itself are named after the first program in the project file unless otherwise specified.

6 6. Contextual Help

Ui allows the user to right-click on a function name and jump to the function's definition by means of a popup menu. The right-click event triggers a popup menu to be displayed with the correct link to either built-in or user defined function names.

| | Figure 8: A popup menu for a user defined function. Jumps to the line where it is defined.

| | Figure 9: A popup menu for a built-in function. Opens the UTR8 on the line where it is described .

Additionally, if it is not a word, or it is not a known function, a popup menu briefly decribes the right-click functionality.

Currently this feature is limited to exact name variables. An instance of a class or function name would not be recognized as a known function e.g.

```
x := foo()
```

Future work may include expanding the range of known functions to include these additional instances.

Other possible additions may be an extension of the tooltip class to include a subclass for right-click events which would be utilized here in place of the descriptive popup menu. This would allow for a more detailed description.

Having the word under the cursor defined allows for the possible addition of a highlighting feature or a "find all usage" feature.

7 7. Ui Options

Ui supports command-line options to be passed to wicont with the Compiler Options... menu item in the Compile menu. Command-line arguments passed to the Unicon program when it is run are supplied via the Arguments... menu item in the Run menu.

The IDE supports the ability to modify the desired font and make changes to the Ui layout and color scheme. The visible view-ports can be adjusted by selecting Window from the View menu; furthermore the tool bar can be displayed/hidden by choosing the appropriate option from within same View menu. The font settings and the Ui preferences can be accessed from the Config menu by selecting Font and Preferences respectively.

| | Figure 10: the View Menu

From the Preferences dialogue, parentheses matching can be disabled and various colors can be adjusted for a desired look.

| | Figure 11: the Preferences Dialog

Once a desired look and layout have been achieved, the above mentioned settings, as well as the position of the Ui and the window width/height can be saved under the Config menu. These settings may manually be specified in a file called _uirc that is read when the IDE starts up. The _uirc resides in the current Ui directory, unless a specified HOME or LOCALAPPDATA environment variable is set, in which case it is taken to be the path-name of the initialization file that Ui is to use. An example _uirc file follows:

```
position=105,23
width=1770
height=1050
window_setup=view_window_all
toolbar_status=show
msglines=7
font=Comic Sans MS,24,italic,bold
linebreak=LF
val_background_color=white
linenumber_highlight_color=white
default_text_color=black
cursor_highlight_color=black
cursor_text_color=white
syntax_text_color=dark-blue-green
glob_text_color=dark-green
```

```
procedure_text_color=purple
quote_text_color=light-brown
comment_text_color=blue
linenumber_text_color=grey
text_highlight_color=light-grey
error_text_color=red
parentheses_match=1
```

8 8. History and Related Work

Many Bothan spies died to bring us this information. – Mon Mothma

In the early 1990's, a prominent member of the Icon community (Bob Goldberg, if I recall correctly) did a rough prototype IDE for Icon on Windows, which whetted our appetite for this category of tool. An M.S. student at UTSA, Steve Schiavo, did a second prototype IDE in Borland Delphi. These efforts featured basic editing, compiling, and executing but were not feature complete and never released.

Clint Jeffery ported Icon's graphics facilities to Windows at UTSA, as part of his agreed contribution for the book "Graphics Programming in Icon"[3]. An IDE (along with a setup.exe installer) was considered essential for the Windows Icon distribution. In order to minimize external language- and compiler-dependencies, and in order to test and demonstrate the functionality of Windows Icon, Jeffery implemented an IDE similar to Steve Schiavo's in Icon as a program called Wi (Windows icon). In order to provide an attractive, native-looking GUI, Windows95 native dialogs (the Win* functions) were added to the language specifically for use in Wi. The Wi program was modified slightly to form Wu (Windows unicon) when that language was invented.

Although the Wu program was nice, it was limited to only run on Windows, and made extensive use of Windows-only native GUI functions. Even on Windows it had some nasty limitations, such as a 32kb file size limit imposed by the classic native windows text editor widget. After Robert Parlett contributed his wonderful GUI class library, which supported fonts and colors, it became ever more desirable to build a Unicon IDE that would run on all Unicon platforms. Bryan Ross's RUDE (Ross' Unicon Development Environment) was an attempt to address the multi-platform requirement in prototype form.

Finally, the Ui program was developed by Clint Jeffery, using Parlett's IVIB interface builder and adapting code from Wu. Unfortunately, various elements of Wu were not replicated immediately in Ui, and it has only gradually progressed towards a complete or usable feature set. Its toolbar was added by Nolan Clayton, who also made various improvements to its visual appearance. Syntax coloring was added by Luis Alvidrez. Hani bani Salameh ported it to version 2 of the GUI classes. It is by now the work of several people, and remains very small and lacks features compared with mainstream IDE's. However, it continues to evolve.

9 9. Implementation Notes

UI has grown slowly to the point where it is now somewhat complicated for new developers to learn and contribute to it. This section is fragmentary and may eventually be completed and split out into a separate document, but for now, here is what the implementers have contributed.

The contextual help on right-click feature was added by defining the word under the cursor, creating tables for known functions, searching the tables for a word, incorporating jump-to methods and a popupmenu on right-click.

The following is the framework for the use case of this functionality

| | Figure 12: Activity Diagram For Right-Click Functionality.

9.0.1 9.1.) Word Under Cursor

The first thing that happens after a user right-clicks within the Ui Editor is the text under the cursor is compared against the definition of a word. A word is defined as a set of characters not containing a specific character: whitespace, tab, period, etc. It expands its collection of characters in both directions, starting with the character directly under the cursor, and continues until it finds one of those characters.

The code which collects the word under the cursor is described here:

```
editBuffer := ide.CurrentEditBox().get_contents()
line := editBuffer[ide.CurrentEditBox().cursor_y] | []
x := ide.CurrentEditBox().cursor_x | []
non_whitespace := &cset -- '            t' -- '(' -- '.'

if any(non_whitespace,line,x) then {
```

```
   while x > 1 & any(non_whitespace,line,x-1) do {
      x -:= 1
      }
   end_of_word := many(non_whitespace,line,x)
   the_word := line[x:end_of_word]
   }
```

The function returns the word or a -1 if it was not a word.

9.0.2 9.2.) Known Functions

The known functions are defined in two tables. The built-in functions are loaded into a table from a list of function names. The user defined functions are loaded into a table by means of a recursive call to the classbrowser tree which contains the function names and line numbers of that project file. The call to the recursive function is here:

```
method treeRecursion(N)
local t
   every t := 0 to *N.subnodes do {
      treeRecursion(N.subnodes[t])
      }
   if not (N.label[-4:0] == ".icn") then {
      if not member(usr_func_table, N.label) then
      insert(usr_func_table,N.label, N.lineno)
      }
end
```

9.0.3 9.3.) Function Look Up

Once a word is defined, it is looked for in the known function tables. If it is a match it is sent to a jump-to function.

9.0.4 9.4.) Jump-To Definitions

Depending other whether the function name was a user defined or a built-in, it will call a jump-to method for that respective type.

The built-in jump-to function utilizes a method which opens a browser at a specified web address. In this case it opens the unicon technical report 8, which contains definitions of all the built-in functions and their uses. Each function name contains an anchor which will allow the browser method to open the web page at the correct location.

If no internet connection is available, it loads up the UTR8 from the source file on the computer.

If it is a user defined function it moves the cursor to the line number where the function is defined, if it is located within the current project file.

10 10. Conclusions and Future Work

The Ui program is a relatively small multi-platform IDE for Unicon. Its original author (C. Jeffery) is grateful for the help he has received from many persons improving Ui and making it less of a toy. Ui remains perpetually in need of more and better "polish", and the list of planned features is long.

- integration of IVIB and Ui – like other languages' IDE's.
- Emacs mode – ground work for programmable keybindings has been laid.
- integration of UDB

11 References

1. Clinton Jeffery, Shamim Mohamed, Ray Pereda, and Robert Parlett,

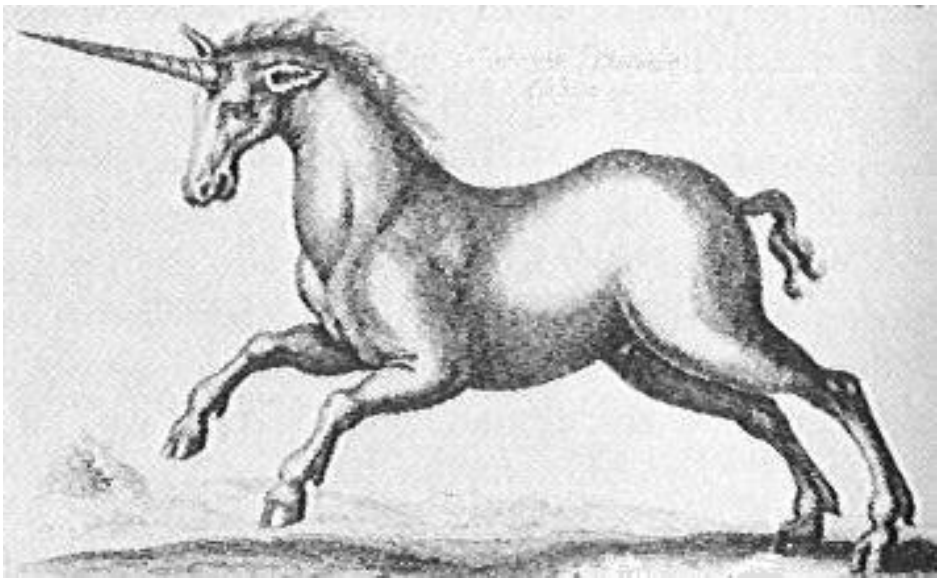
Programming with Unicon, http://unicon.org/ub/ub.pdf, 2005.

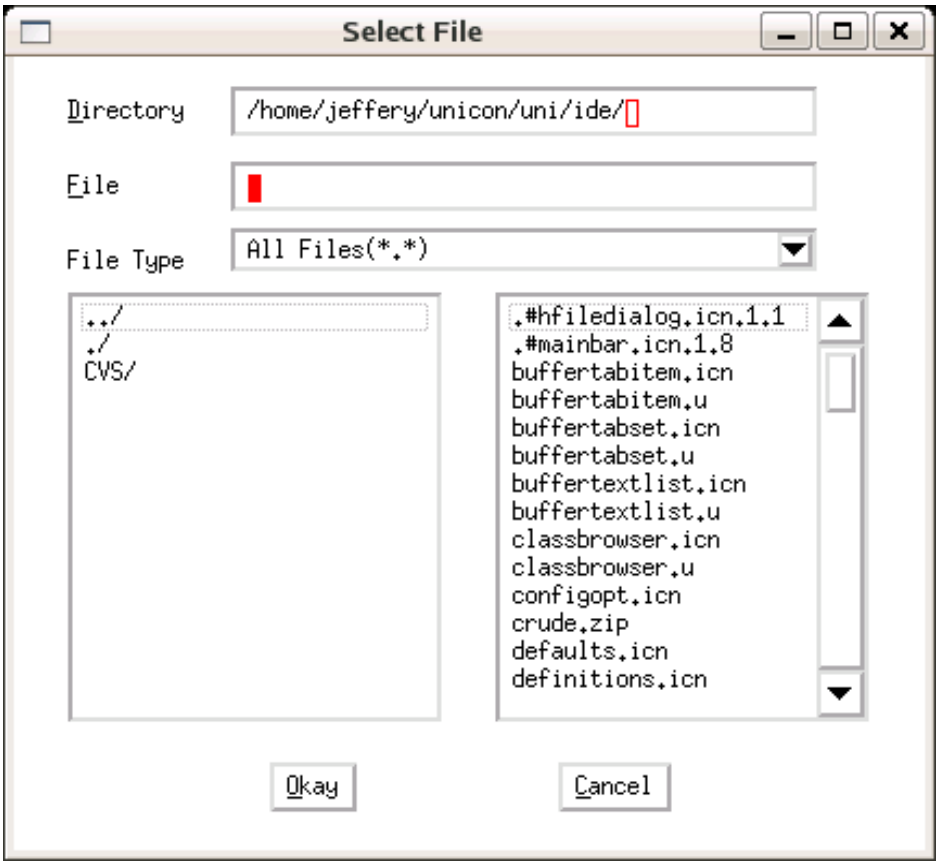
1. Clinton Jeffery, Version 11 of Unicon for Microsoft Windows,

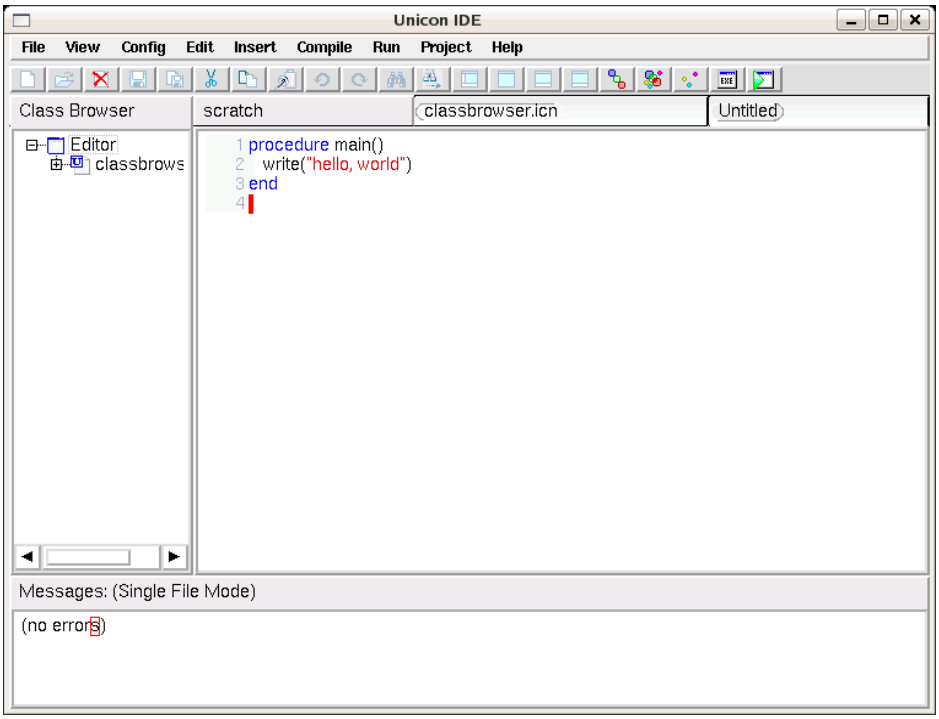
http://unicon.org/utr/utr7.html, 2006.

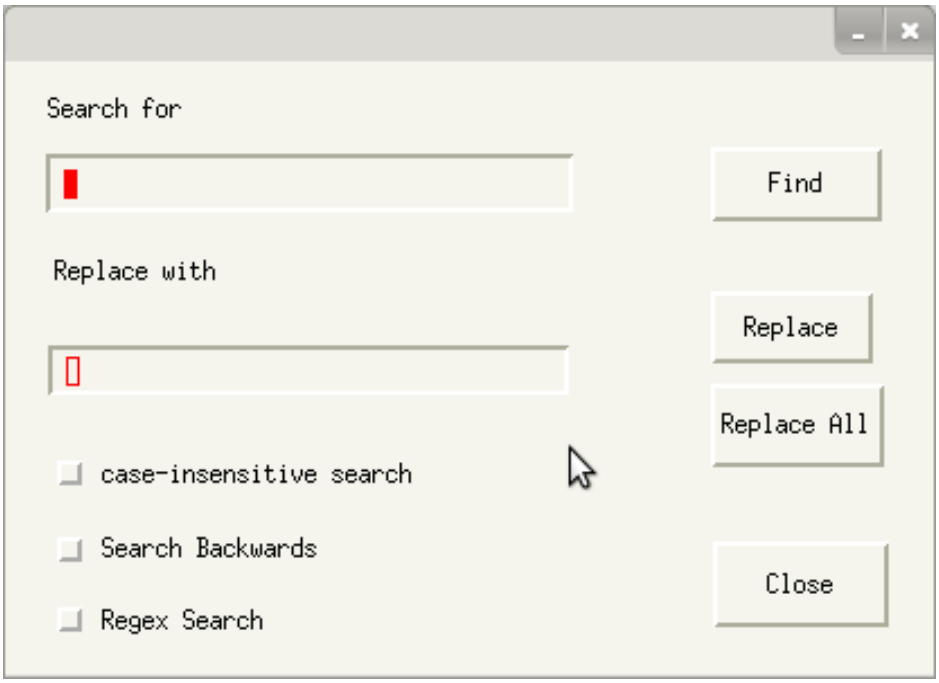
1. Gregg M. Townsend, Ralph E. Griswold, and Clinton L. Jeffery,

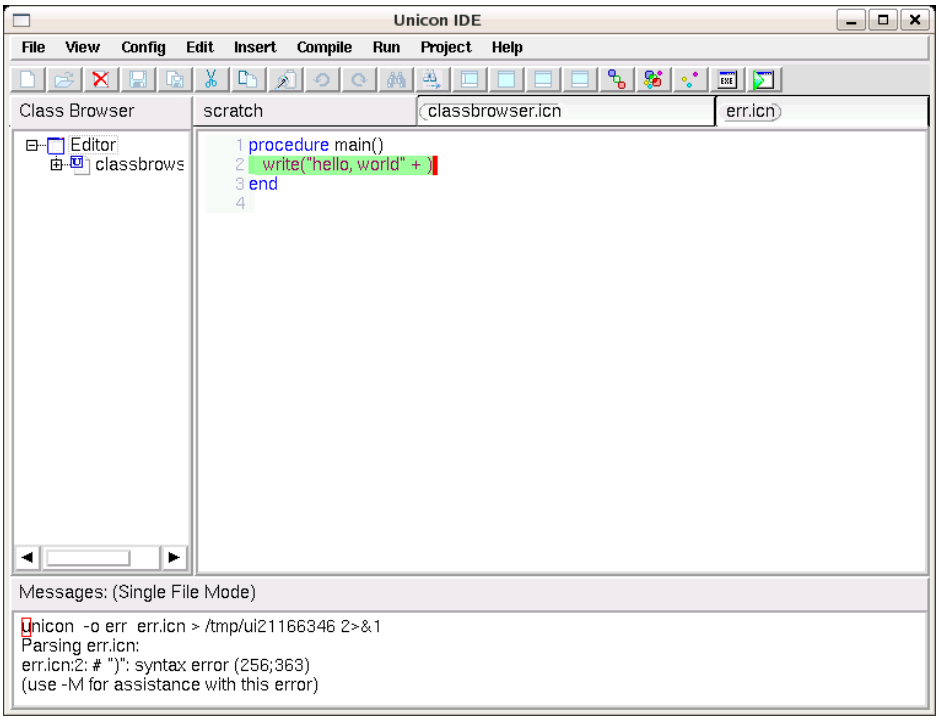
Graphics Facilities for the Unicon Programming Language Version 9.3, The Univ. of Arizona Icon Project Document IPD281, 1996.

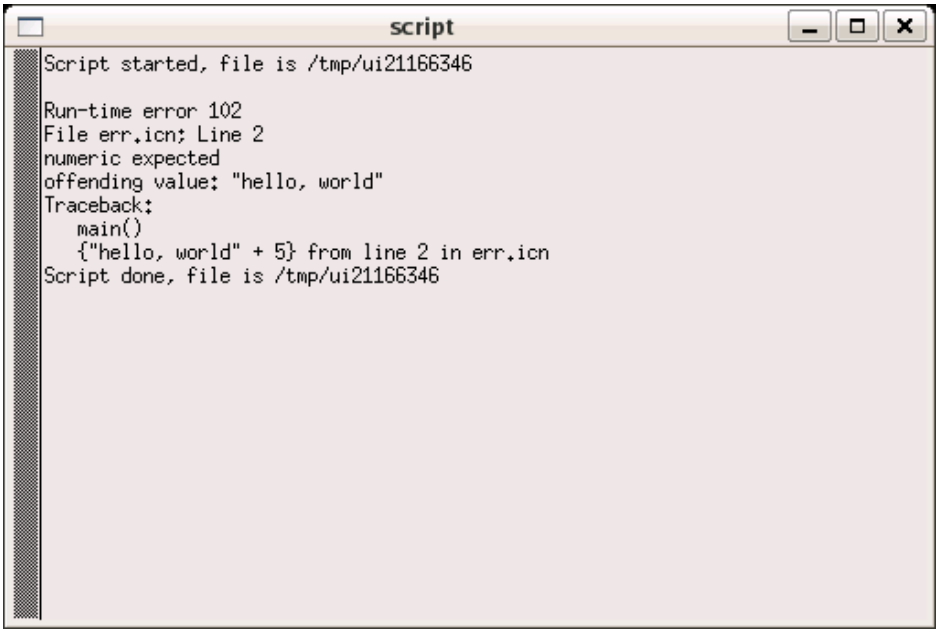


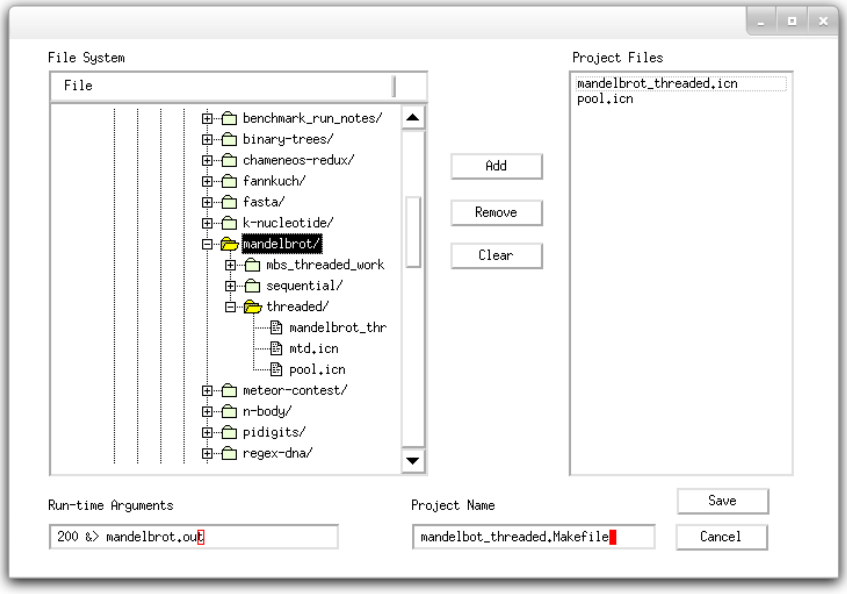

Select File
Directory
/home/jeffery/unicon/uni/ide/
File
File Type
All Files(*.*)
../
./
CVS/
.#hfiledialog.icn.1.1
.#mainbar.icn.1.8
buffertabitem.icn
buffertabitem.u
buffertabset.icn
buffertabset.u
buffertextlist.icn
buffertextlist.u
classbrowser.icn
classbrowser.u
configopt.icn
crude.zip
defaults.icn
definitions.icn
Okay
Cancel

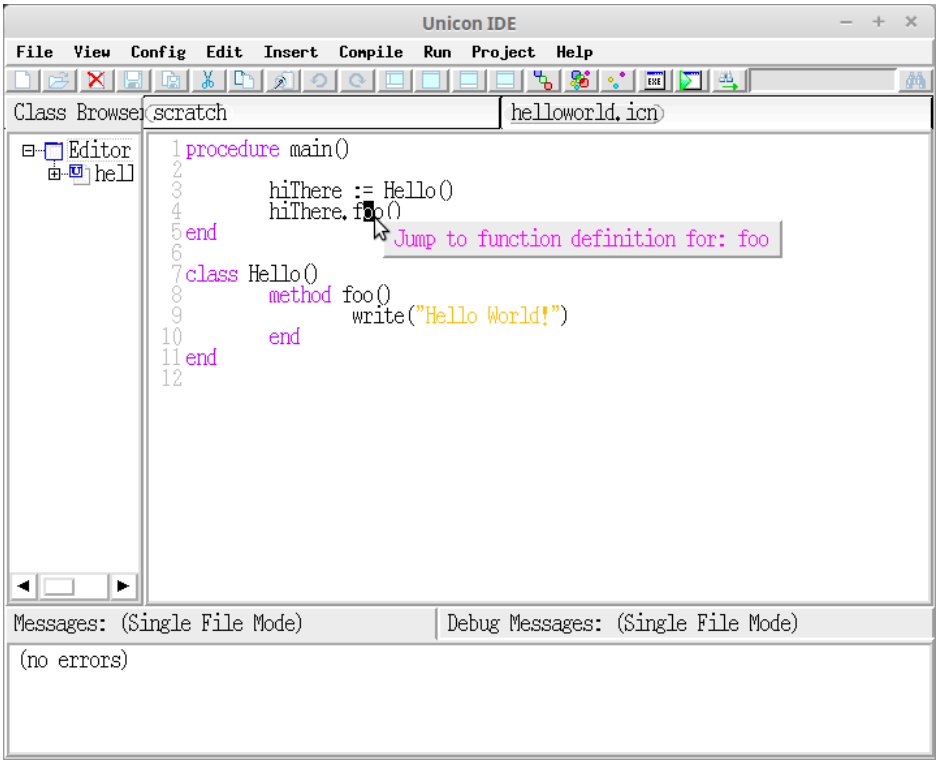

Unicon IDE
File View Config Edit Insert Compile Run Project Help
Class Browser
scratch
classbrowser.icn
Untitled
Editor
classbrows
1 procedure main()
2 write("hello, world")
3 end
4
Messages: (Single File Mode)
(no errors)

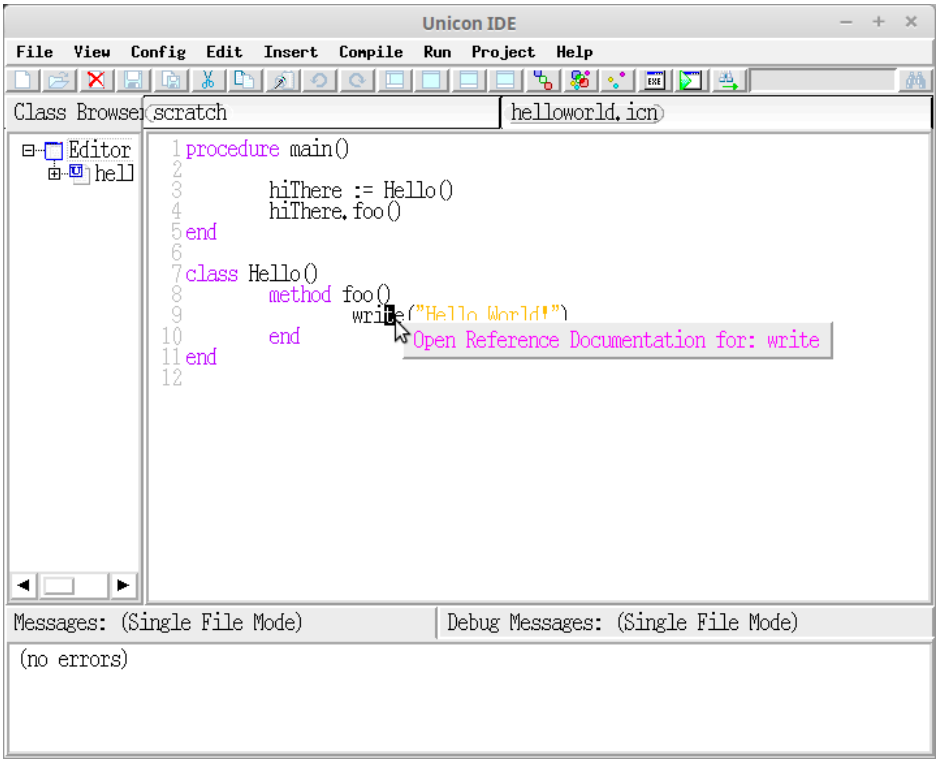

Search for
Find
Replace with
Replace
Replace All
case-insensitive search
Search Backwards
Regex Search
Close

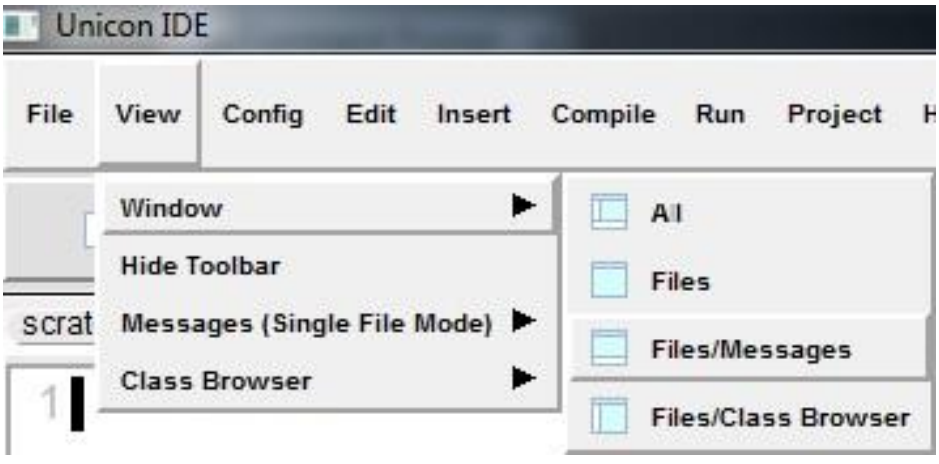

Unicon IDE
File View Config Edit Insert Compile Run Project Help
Class Browser
scratch
classbrowser.icn
err.icn
Editor
classbrows
1 procedure main()
2 write("hello, world" + 5)
3 end
4
Messages: (Single File Mode)
unicon -o err err.icn > /tmp/ui21166346 2>&1
Parsing err.icn:
err.icn:2: # ")": syntax error (256;363)
(use -M for assistance with this error)

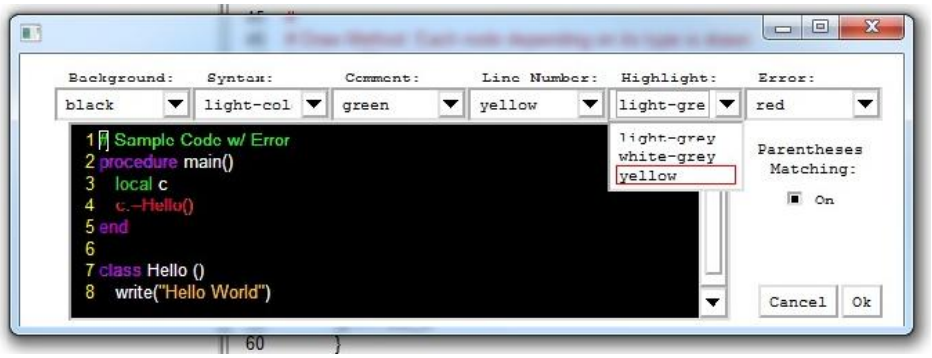

script
Script started, file is /tmp/ui21166346
Run-time error 102
File err.icn; Line 2
numeric expected
offending value; "hello, world"
Traceback;
main()
{"hello, world" + 5} from line 2 in err.icn
Script done, file is /tmp/ui21166346

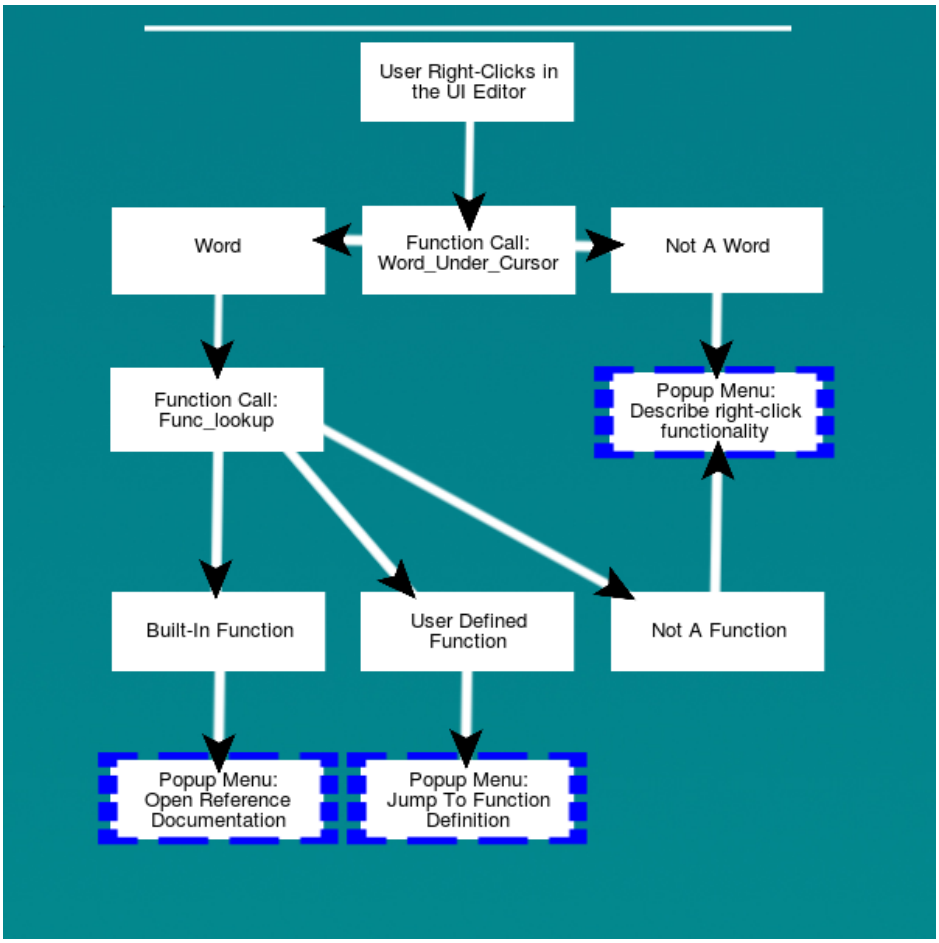

File System
File
benchmark_run_notes/
binary-trees/
chameneos-redux/
fannkuch/
fasta/
k-nucleotide/
mandelbrot/
mbs_threaded_work
sequential/
threaded/
mandelbrot_thr
mtd.icn
pool.icn
meteor-contest/
n-body/
pidigits/
regex-dna/
Add
Remove
Clear
Project Files
mandelbrot_threaded.icn
pool.icn
Run-time Arguments
200 &> mandelbrot.out
Project Name
mandelbot_threaded.Makefile
Save
Cancel


Unicon IDE
File View Config Edit Insert Compile Run Project Help
Class Browser
scratch
helloworld.icn
Editor
hell
procedure main()
hiThere := Hello()
hiThere.foo()
end
Jump to function definition for: foo
class Hello()
method foo()
write("Hello World!")
end
end
Messages: (Single File Mode)
Debug Messages: (Single File Mode)
(no errors)


Unicon IDE
File View Config Edit Insert Compile Run Project Help
Class Browser
scratch
helloworld.icn
Editor
hell
1 procedure main()
2
3 hiThere := Hello()
4 hiThere.foo()
5 end
6
7 class Hello()
8 method foo()
9 write("Hello World!")
10 end
11 end
12
Open Reference Documentation for: write
Messages: (Single File Mode)
Debug Messages: (Single File Mode)
(no errors)


Unicon IDE
File View Config Edit Insert Compile Run Project
Window
Hide Toolbar
Messages (Single File Mode)
Class Browser
AI
Files
Files/Messages
Files/Class Browser


Background:
Syntax:
Comment:
Line Number:
Highlight:
Error:
black
light-col
green
yellow
light-gre
red
1 -- Sample Code w/ Error
2 procedure main()
3 local c
4 c.~Hello()
5 end
6
7 class Hello ()
8 write("Hello World")
light-grey
white-grey
yellow
Parentheses Matching:
On
Cancel
Ok


User Right-Clicks in the UI Editor
Word
Function Call: Word_Under_Cursor
Not A Word
Function Call: Func_lookup
Popup Menu: Describe right-click functionality
Built-In Function
User Defined Function
Not A Function
Popup Menu: Open Reference Documentation
Popup Menu: Jump To Function Definition